

Trainer Guide: Spring Boot Session 2 - REST Web Services

A slide-by-slide read-aloud script for [springboot-session2-theory.html](#). Everything on screen, explained in plain spoken language.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [springboot-session2-theory.html](#) (20 slides)

Each slide has an ON SCREEN reminder and a SAY section you can read out word-for-word. EMPHASISE and DELIVERY TIP notes are for you only - do not read those aloud.

Slide 1 - Title: REST Web Services

ON SCREEN

The blue cover slide: 'Spring Boot', with the line about building a complete Student CRUD API.

SAY (read aloud)

Welcome back to Session 2. In Session 1 we got an application running and returning a greeting. Today we make it talk - we turn it into a REST web service that exchanges JSON data with any client. The theory today is deliberately shorter than last time, because the real weight of this session is the hands-on lab, where you'll build a complete Student CRUD API yourself - all four operations, endpoint by endpoint. Let's set up the concepts you'll need, then go build it.

Slide 2 - Agenda

ON SCREEN

Six agenda items: Session 1 recap, web services and JSON, REST vs SOAP and HTTP, URI design and annotations, project setup, and the hands-on lab.

SAY (read aloud)

Here's the plan. We start with three quick warm-up questions from Session 1. Then web services and JSON - what it means to exchange data instead of web pages. Then REST versus SOAP, the HTTP methods, and status codes. Then URI design and the annotations you'll use today. A quick word on project setup with DevTools and Swagger. And then the main event - building the Student CRUD API live in the lab. Keep the bigger picture in mind: by the end you'll be able to explain what a REST API is and build one from scratch with the standard Spring Boot annotations.

Slide 3 - Session 1 recap

ON SCREEN

Three click-to-reveal flash cards on @SpringBootApplication, @RestController, and the default port.

SAY (read aloud)

Let's warm up with three quick questions - I'll read each, you answer in your head, then we reveal it. Question one: what does @SpringBootApplication do? It's one annotation combining three - Configuration, EnableAutoConfiguration, and ComponentScan - and it marks the entry-point class that boots the whole app. Question two: what is @RestController? It marks a class as a web request handler whose return values are written directly into the HTTP response body as JSON or text, rather than being treated as the name of a page to render. Question three: what port does Spring Boot start on by default? Port 8080 - the embedded Tomcat starts there automatically, and you change it with server dot port.

DELIVERY TIP

Treat these as rapid verbal call-and-response to re-energise the room. Click each card only after a couple of people answer.

Part 1 - Web Services & Data

Slide 4 - What is a web service?

ON SCREEN

A definition, plus a two-column diagram contrasting a traditional web app returning HTML with a web service returning JSON.

SAY (read aloud)

Here's the core idea of today. A web service is a way for two applications to exchange data over a network, regardless of the language or platform either one is written in. The key word is data. Look at the contrast on screen. A traditional web app, on the left, returns HTML - a page meant for a human's eyes in a browser. A web service, on the right, returns data - JSON or XML - meant for another program to consume. Same server technology underneath, completely different audience. So when we build a REST API today, we're building the right-hand side: something that hands structured data to a mobile app, a front-end, or another service - not a web page.

Slide 5 - JSON syntax

ON SCREEN

A single JSON object for a student, then a JSON array of students, with the parts colour-coded.

SAY (read aloud)

JSON - JavaScript Object Notation - is the format your API will send and receive, so let's make it land. Look at the single student object. Curly braces wrap one object - one record. Inside are key-value pairs: the key is always a quoted string, then a colon, then the value. Commas separate the pairs. Values can be strings in quotes, bare numbers, or booleans - true and false. The second example shows square brackets, which make an array - an ordered list. So a response with several students is an array of objects. And here's the good news - you won't write this by hand. Spring Boot uses a library called Jackson to automatically convert your Java objects into JSON on the way out, and JSON back into objects on the way in. That conversion is called serialization.

Slide 6 - Key web service terminology

ON SCREEN

Four cards with analogies: service provider, service consumer, synchronous, asynchronous. A key box on loose coupling.

SAY (read aloud)

Four terms come up constantly, each with an everyday analogy so it sticks. The service provider is the application that offers the service and answers requests - the kitchen that cooks the food; your API is the provider today. The service consumer is the application that calls it and uses the data - the customer who orders; Postman or a browser are consumers. Synchronous means the consumer waits for the response before doing anything else - standing at the counter until your order arrives. Asynchronous means it doesn't wait - it carries on and is notified later, like taking a buzzer and sitting down.

EMPHASISE

Land the superpower in the key box: because the two sides only agree on the data format and the URLs, neither cares what language the other is written in. A Python app, a React front-end, and an Android phone can all call the same Java API. That independence is called loose coupling, and it's why web services scale across teams and technologies.

Part 2 - REST, HTTP & Design

Slide 7 - REST vs SOAP

ON SCREEN

A comparison table of SOAP versus REST across what it is, data format, weight, complexity, contract, and transport.

SAY (read aloud)

There are two main ways to build web services. SOAP is the older, heavier, XML-based standard. REST is the lighter, more flexible style that dominates modern APIs - and the one we'll use. SOAP isn't the villain; it's just from an earlier era. Read across the table: SOAP is a strict protocol with rigid rules; REST is an architectural style with guidelines. SOAP is XML-only; REST usually uses JSON. SOAP is heavier and more verbose; REST is lightweight and fast. SOAP requires a formal contract file called a WSDL; REST doesn't. In industry, REST is the standard for web, mobile, and microservices. SOAP still lives in legacy banking and large enterprise systems where its strict contracts were valued. You'll meet REST everywhere; SOAP occasionally in older codebases.

Slide 8 - The six REST principles

ON SCREEN

A list of the six REST constraints: stateless, client-server, uniform interface, cacheable, layered system, code on demand.

SAY (read aloud)

REST is defined by a set of constraints. You don't need to memorise all six today, but knowing they exist helps you understand why REST APIs are built the way they are. Stateless - every request carries everything the server needs; the server keeps no memory between requests. Client-server - the UI and the data are separate and evolve independently. Uniform interface - a consistent, predictable way to access resources. Cacheable - responses can declare whether they may be stored and reused. Layered system - there can be intermediaries like load balancers and gateways without either side knowing. And code on demand, the one optional constraint, rarely used.

EMPHASISE

If they remember only one, it's stateless. Each request stands alone; the server doesn't remember who you are between calls. That single principle is why REST APIs handle huge traffic gracefully.

Slide 9 - HTTP methods map to CRUD

ON SCREEN

A table mapping GET, POST, PUT, DELETE to Read, Create, Update, Delete with Student API examples.

SAY (read aloud)

REST uses the standard HTTP methods as verbs describing what you want to do to a resource, and the four you'll use map directly onto the four database operations known as CRUD - Create, Read, Update, Delete. GET reads existing data - GET slash students lists all students. POST creates new data - POST slash students adds a new student. PUT updates existing data - PUT slash students slash id updates one. DELETE removes data - DELETE slash students slash id. Here's the mental model: think of a student record. You GET it to view, POST a new one when a student enrolls, PUT to fix their email, and DELETE when they leave. Same noun - slash students - a different verb for each action. That's the whole idea of a REST API.

Slide 10 - HTTP status codes

ON SCREEN

Colour-coded badges: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error.

SAY (read aloud)

Every HTTP response carries a numeric status code telling the client what happened. There are dozens, but you only need five today. They're grouped: 2-hundreds mean success, 4-hundreds mean the client made a mistake, 5-hundreds mean the server broke. 200 OK - the request worked; Spring returns this by default for a successful GET. 201 Created - used after a successful POST; you'll set it yourself with `at-ResponseStatus`. 400 Bad Request - the client sent something malformed, like invalid JSON. 404 Not Found - the URL or resource doesn't exist, like asking for a student ID that isn't there. And 500 Server Error - something threw an unhandled exception in your code; we'll learn to handle these gracefully in Session 3.

Slide 11 - URI design principles

ON SCREEN

A good-versus-bad comparison: noun-based URLs versus verbs-in-the-URL, and a key box on the pattern.

SAY (read aloud)

A URI - the path part of a URL - names a resource. The golden rule of REST: URIs are nouns, HTTP methods are verbs. Don't put the action in the URL - let the method say what you're doing; the URL just names what you're acting on. On the good side: slash students for the collection, slash students slash id for one student, and a query parameter like question-mark name equals John for filtering. On the bad side: slash getStudent, slash deleteStudentById, slash student slash fetch - the verb does not belong in the path. The pattern to internalise: plural nouns for collections, a path segment to identify one item, and query parameters for optional filtering. The verb is never in the URL.

Slide 12 - Annotations you'll use today

ON SCREEN

A grid previewing ten annotations, from `@RestController` to `@ResponseStatus`.

SAY (read aloud)

Here's your whole toolkit for the lab, in one place. Don't worry about the details yet - we'll unpack the important ones, and you'll use each in code. Just see what's coming. `AtRestController` and `atRequestMapping` set up the controller and its base path. `AtGetMapping`, `atPostMapping`, `atPutMapping`, and `atDeleteMapping` map the four verbs. `AtPathVariable` pulls a value from the URL path; `atRequestParam` pulls a query parameter; `atRequestBody` turns the incoming JSON into a Java object. And `atResponseStatus` sets the HTTP status code. Ten annotations - that's the entire surface area of building a REST controller. You already know `atRestController` from Session 1.

Slide 13 - `@RestController` unpacked

ON SCREEN

A note that `@RestController` equals `@Controller` plus `@ResponseBody`, with two code panels comparing them.

SAY (read aloud)

You used `atRestController` in Session 1, but here's what it actually is under the hood. It's a convenience annotation bundling two others: `atController` plus `atResponseBody`. `AtResponseBody` is the important half. It tells Spring to take whatever your method returns and write it directly into the HTTP response body as JSON, via Jackson, instead of treating the return value as the name of an HTML page to render. Look at the panels. With a plain `atController` you'd have to add `atResponseBody` to every single method. With `atRestController` it applies to all methods automatically. That's why we use it for APIs - every method returns data, not a page.

Slide 14 - @RequestMapping at class level

ON SCREEN

A before/after showing repeated /students paths collapsing into a class-level prefix, and a key box on URL assembly.

SAY (read aloud)

Putting at-RequestMapping slash students on the class sets a common URL prefix for every method inside. Each method's mapping is then relative to that prefix. This keeps your code clean - you write slash students once instead of repeating it everywhere. Look at the before-and-after: without it, every method repeats slash students in its path. With the class-level mapping, the methods just declare the suffix - an empty mapping, or slash all, or slash id. How the URL is assembled: Spring joins the class prefix and the method path. Class slash students plus method slash id gives the full endpoint slash students slash id. Change the prefix in one place and every endpoint moves with it.

Part 3 - Tools & The Bigger Picture

Slide 15 - Testing tools: Postman & Swagger

ON SCREEN

Two cards - Postman and Swagger UI - plus a mock Swagger listing of endpoints.

SAY (read aloud)

You can't easily test a POST or PUT from a browser address bar - browsers only do GET. So we use proper API testing tools, and you'll use both today; they complement each other. Postman is a dedicated desktop app for calling APIs. You get full control: pick the method, set headers, paste a JSON body. It's the industry-standard tool teams use daily, and it's best for testing POST, PUT, and DELETE with custom bodies. Swagger UI is a browser-based tester that's auto-generated from your annotations - zero setup. Add one dependency and Spring builds a live, interactive page listing every endpoint with 'Try it out' buttons. Great for quick checks and sharing. The mock on screen shows what that endpoint list looks like - it builds itself from your controller.

Slide 16 - ResponseEntity

ON SCREEN

Two code panels: a raw return that is always 200, versus ResponseEntity that lets you choose the status.

SAY (read aloud)

By default, when you return a Student from a method, Spring sends it back with status 200 OK. But sometimes you need control over the full response - the status code, the headers, and the body. That's what ResponseEntity gives you: it's a wrapper around the entire response. Look at the panels. On the left, a raw return - the status is always 200, even if the student wasn't found. On the right, ResponseEntity - you return ResponseEntity.ok with the student for 200, or ResponseEntity.notFound for 404, depending on the outcome. Reach for ResponseEntity when you need different status codes depending on what happened - 201 Created after a POST, 404 when something's missing, or to attach custom headers. It's the professional way to build precise REST responses.

Slide 17 - RestTemplate

ON SCREEN

A RestTemplate code snippet calling another service, with bullets and a 'trainer demo only' warning.

SAY (read aloud)

So far your app is the provider - it answers requests. But sometimes a Spring Boot app needs to be a consumer too, calling another REST API from inside its own code. RestTemplate is Spring's classic synchronous client for exactly that. In the snippet, getForObject sends a GET request and converts the JSON response directly into a Java Student object. It's synchronous - your code waits for the response before continuing. And there are matching methods: postForObject, put, delete, and more.

DELIVERY TIP

This is a trainer demo only - don't have them code it today. Note it becomes far more relevant in a microservices context, where many small services call each other, which is the Session 4 topic.

Slide 18 - Coming up in Session 3

ON SCREEN

Four cards previewing Session 3: exception handling, JPA plus H2, @Entity, @Repository.

SAY (read aloud)

A look ahead. Right now today's data lives in a list in memory - it vanishes when the app stops, and asking for a missing ID just returns nothing useful. Session 3 fixes both. We'll add proper exception handling, returning clean 404 messages instead of ugly stack traces. We'll swap the in-memory list for a real H2 database with Spring Data JPA, so data survives restarts. We'll map our class straight to a database table with @Entity. And we'll get real CRUD with almost no boilerplate using a @Repository. Everything you build today carries forward - Session 3 upgrades it from a demo into a real, persistent application.

Slide 19 - Annotations recap

ON SCREEN

The same ten annotations from the preview, now each with a one-line definition.

SAY (read aloud)

Here's your closing cheat-sheet - the same ten annotations, now with a one-line definition each. Keep this slide as your revision anchor; it's the entire vocabulary of building a REST controller in Spring Boot. At-RestController marks the controller and auto-serializes returns to JSON. At-RequestMapping sets a prefix or path. At-GetMapping, PostMapping, PutMapping, and DeleteMapping map the four verbs. At-PathVariable binds a path segment; at-RequestParam binds a query parameter; at-RequestBody deserializes the request body JSON into an object. And at-ResponseStatus sets the HTTP status code the method returns.

Slide 20 - Closing

ON SCREEN

The closing blue slide: 'Let's build it.'

SAY (read aloud)

That's the theory for Session 2. You can now explain what a web service is, how REST models resources with nouns and verbs, what the status codes mean, and which annotations build a controller. Now let's open the lab sheet and write the Student CRUD API together, endpoint by endpoint. We'll build it the same way real teams do - one operation at a time, testing each in Postman as we go. Let's build it.